Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I expect to use loops to iterate through dictionaries and lists when looking up

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The easiest parts are creating the global constant dictionaries and lists of the notes and templates and chord fingerings. The hardest part will be making the chord diagrams into more complex aesthetic diagrams since it will be hard to create a photo specific to not only each chord, but also each fingering.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

The most time will likely go to creating the SVG diagrams for each chord. I will start with the state class and the homepage and a simple result page containing a

progression in list form without the diagram to assure the idea works.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

The courseload on top of other classwork may distract me. But I will be fine as long as I manage my time well.

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.

Home Page

Progression Generator

Key ☐ ← dropdown
Mood ☐ ← dropdown
Length ☐ ← dropdown
7th Chords ☐ ← checkbox
Beginner ☐ ← checkbox

[Generate] ← button
[saved progressions] ← button
[Mood guide] ← button

Result Page

summary:
Joyful Progression in G

Pattern:
I-V-vi-IV
Parent Scale:
G, Am, Bm, C, D, Em
Chords:



D Em C

...

[detail] ← button

Transpose: [+] [-] ½ step
↑ ↑ buttons

[Regenerate] ← button

[save] ← button

[Back] ← button

Chord Detail

G Major:



Notes:

G, B, D, G, D, G

[sound] ← button (plays sound)

[Back] ← button to result page

---

saved Progressions

Nothing saved yet ← if true

- name, mood, key

G - D - Em - C

[Delete] [load]
↑ buttons ↑ brings to result page

Mood Guide

Explains each mood selection

[Back to Home]
↑ button

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	My site will allow users to input various choices about a guitar progression they want to generate, and will generate it. They will be shown information about the generation such as the chords and diagrams. They will be able to save progressions and see more information about each chord and about each mood category.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	class State: key: str #chosen key different with each progression mood: str #chosen mood different with each progression length: int #chosen length different with each progression use_sevenths: bool transpose_steps: int #shows the user how many steps they have transposed as they increase or decrease it current_progression: list[Chord] #updated as for each now progression saved_progressions: list[Progression] #updated each time a new progression is saved
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	class Chord: name: str root: str quality: str #for example, maj7 or m7 chord frets: list[int] is_barre: bool

	This class is for a single chord. Each chord will be generated in accordance with the key and the progression pattern chosen by the mood. So the name, root note, quality, frets, will all differ for each chord. class Progression: name: str mood: str key: str chords: list[Chord] This class is for a saved chord progression. The name can be possibly customized, and the mood and key will already exist. The progression will be a list of chord instances.
Routes (minimum 7) The names of your route functions	index, generate, show_result, transpose_up, transpose_down, chord_detail, save_progression, view_saved, delete_saved, mood_guide, regenerate, load_saved.
Pages (minimum 5) A brief description of the pages in your site	Home/Setup, Result, Chord Detail, Saved Progressions, Mood Guide.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	The key, mood, and length selections on the homepage will all have selectbox choices. The 7th chord option and the beginner option (decided whether or not to include barre chords) will have checkbox. There may be a textbox on the result page where the user can name the progression before saving it.

If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	Minor keys will be handled by if statements. If a minor key is selected the starting note will be changed to map it to its corresponding major scale. If statements will also be used for both of the selectbox. If 7th is selected, then the program will include 7ths in the progression. If beginner is selected, then barre chords will be excluded. If there are no saved lists, the saved page will read a message saying there are no saved lists. If the name already exists for a saved progression, an error will read that name is in use, If bluesy mood is selected a special 12-bar progression will be generated regardless of other inputs.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	The diagrams will be drawn by looping over the list of saved progressions. The list of saved items will be printed by looping over the list of saved items.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Create a word doc of all the requirements for the project adn how each one will be met with this idea

I'll build a Word document that maps each project requirement to how your chord-progression app meets it.

I made a five-page Word document, Final_Project_Requirements_Plan.docx. It lists each requirement from the assignment and how your chord progression app will meet it. I opened the rendered pages and checked the first and third. I didn't look at the other three.

It covers:

- ***Site functionality:*** *routes, pages, State fields, the second dataclass and list of instances, inputs,* `if` *statements, loops, constants instead of globals, and documented advanced techniques.*
- ***Code quality:*** *tests, docstrings, type hints, the 120-character limit, and helper functions.*
- ***Repository:*** *planning document, deployment,* `set_site_information`*, commit messages, the video, and AI conversations.*
- ***Supporting sections:*** *key design decisions, State and dataclass fields, a week-by-week schedule, and risks.*

The document assumes five pages: Home, Result, Chord Detail, Saved Progressions, and Mood Guide. I listed a Compare page as optional. That keeps you at the minimum, so cut any of the five and you fall below the requirement. If you'd rather have a safety margin, make Compare a firm sixth page.

What are my steps for sectioning off the coding of this project?
Build the pure logic first, then one thin working path through the app, and add features one at a time. This is the order Drafter's own "Your Project" guide recommends. Run the app and your tests after every step.

Setup (before any features)

1. *Create* `main.py` *with a bare* `State`*, one route, and* `set_site_information(...)`*.*
2. *Deploy this empty app to GitHub Pages now, so deployment problems show up on day one instead of the last night.*

Logic first (no Drafter needed)

3. ***Constants and dataclasses.*** *Write* `NOTES`*,* `MAJOR_SCALE_STEPS`*,* `MOOD_TEMPLATES`*, a small* `CHORD_FINGERINGS` *(start with about 8 open chords), and the* `Chord` *and* `Progression` *dataclasses.*
4. ***Core helpers, each with a test as you write it.*** *Build them in this order:* `transpose_note`*,* `get_scale`*,* `relative_major`*,* `numeral_to_chord`*,* `build_progression`*,* `lookup_fingering`*.*
5. ***Test the tricky cases now:*** *B up one step wraps to C, A minor starts on Am, and a 7th option changes the chord quality.*

One working path

6. *Build* `index`*,* `generate`*, and* `show_result`*, with the progression shown as*

plain text first. Check that you can click from the setup page to a result.

7. *Add `draw_diagram` and display it with `SVG`. Get one chord drawn correctly (G or C) before looping over a whole progression.*
8. *Put the chords in a wrapping `Row` with a loop over `current_progression`.*

Add features one at a time

9. ***Transpose:** `transpose_up` and `transpose_down`, which update `transpose_steps` and rebuild.*
10. ***Chord detail:** the `chord_detail` route. Read Drafter's `Argument` page first, since that's likely how a button passes a chord name.*
11. ***Save, view, and delete:** `save_progression`, `view_saved`, and `delete_saved`, including the empty-list and duplicate-name `if` checks.*
12. ***Mood guide page** and the rest of the five moods, with Bluesy's 12-bar template last.*
13. ***Full chord coverage:** generate barre chords from the E-shape and A-shape templates, then add beginner mode.*
14. ***Stretch:** `Melody` playback and the difficulty rating, only if everything above works.*

Finish

15. *Write at least one test per route using Drafter's testing functions.*
16. *Do a quality pass: docstrings, type hints, lines under 120 characters, and comments on the advanced techniques.*
17. *Redeploy, record the video, and commit your AI conversations and planning document with descriptive commit messages.*

Two habits will save you the most trouble:

- ***Commit after each step that works.** That gives you clean, descriptive commit messages and a safe point to go back to.*
- ***Don't build the full chord dictionary up front.** Start small, so you can test the whole pipeline early, then expand.*

Want me to write the starter `main.py` for steps 1 to 4, with the constants, dataclasses, and first helpers and tests?

Yes, and do not use overly complicated coding techniques.

Both starter files pass their tests, so I'm sending them now.

I wrote `main.py` (steps 1–4 plus a thin working path) and `test_main.py`. The 7 helper tests pass, but I couldn't install Drafter here, so the Drafter routes were only

checked against a stand-in. Running them for real is your first test.

What's in `main.py`

- ***Constants:*** *`NOTES`, `MOOD_TEMPLATES` for all five moods (Bluesy is the 12-bar blues), and a 22-chord `CHORD_FINGERINGS` starter set.*
- ***Dataclasses:*** *`Chord`, `Progression`, and `State` with 7 fields.*
- ***Helpers:*** *`transpose_note`, `get_scale`, `relative_major`, `numeral_to_chord`, `build_progression`, `lookup_fingering`, and a few small ones. All have type hints and docstrings, and no line goes over 120 characters.*
- ***Routes:*** *`index`, `generate`, and `show_result`. Right now `generate` is hard-coded to Joyful in C so you can click through the whole path.*
- ***Site info:*** *a `set_site_information(...)` call with placeholders. Fill in your name and repo link.*

Things to check on first run

- ***`start_server` guard:*** *I wrapped it in `if __name__ == "__main__":` so the tests can import `main.py`. Confirm the site still launches and deploys. If it doesn't, remove the guard and move the helper tests into `main.py`.*
- ***Real Drafter test:*** *run it once on a fresh install before building more.*

A design gap to decide on

- ***Minor keys:*** *`resolve_key` maps A minor to C major, which works for the Sorrowful and Mysterious templates (they start on vi, so you get Am). Joyful in A minor would still start on C, because the template is major-flavored.*
- ***Options:*** *either only offer minor keys for the minor-feeling moods, or add a rotated template for them.*

Next step
Replace the hard-coded `generate` with real inputs (key, mood, length, and 7ths). Read Drafter's SelectBox and CheckBox pages first, since I didn't pull their exact signatures. Then add `draw_diagram`.

Want me to start on the inputs and diagram next?

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.